FlowForge

Product, UX, Architecture & Delivery Blueprint

A focused workflow automation platform for configuring, executing, observing, and recovering business processes.

Document details	Value
Version	1.1
Prepared	20 October 2026
Status	Proposed product and engineering plan
Primary goal	A credible, production-minded portfolio project with a deliberately limited MVP

Scope principle: build one reliable execution engine and a small set of useful triggers/actions before expanding the visual editor or integration catalog.

1. Executive summary

FlowForge lets a user define a business workflow in a visual editor and have the platform execute it asynchronously. A workflow combines a trigger, a sequence of actions, configuration, and recovery rules. FlowForge persists each execution and each step result so users can understand what happened and recover from failures.

Product promise

Configure the process once, run it repeatedly, and understand every run.

MVP in one sentence

A single-workspace application with webhook, scheduled, and manual triggers; sequential actions; PostgreSQL-backed execution history; a Redis/BullMQ queue; a Node.js worker; retry controls; and a clear UI for creating, publishing, monitoring, and retrying workflows.

What FlowForge is not

- Not a general-purpose programming language or a replacement for Temporal, n8n, Zapier, or enterprise integration platforms.
- Not an arbitrary-code execution environment in the MVP.
- Not a guarantee of exactly-once execution across external systems.
- Not a full enterprise multi-tenant SaaS product on day one.
- Not a full graph editor: the MVP builder is a linear ordered list.
- Known limitation: external side effects are at-least-once. A provider without idempotency support may see a repeated request, and uncertain outcomes are surfaced as outcome_unknown.

Success criteria

- A user can create, test, publish, pause, and inspect a workflow without editing application code.
- Webhook, scheduled, and manual triggers all enter the same execution pipeline.
- A failed action is visible with a useful error and can be retried according to explicit rules.
- Duplicate webhook deliveries do not silently create duplicate executions when an idempotency key is supplied.
- The README and demo explain architecture decisions, failure handling, trade-offs, and how to run the system locally.
- Proposed measurable targets, verified with a documented local benchmark (hardware, dataset size and concurrency recorded in the repository): webhook acknowledged in under 200 ms at p95; 100 concurrent accepted executions complete with no lost accepted executions and no invalid duplicate state transitions; a killed worker is recovered within the lease timeout. External side effects may repeat when a provider lacks idempotency support; attempts are recorded and uncertain outcomes surface as outcome_unknown.

2. Problem, users, and product boundaries

Problem statement

Small teams often hardcode each business automation as a separate endpoint, scheduled script, or background job. Over time, these automations become hard to discover, change, audit, and recover when dependencies fail. FlowForge centralizes workflow definitions and execution visibility.

Primary user

A technical operator, developer, or small-team administrator who understands the business process but wants to configure routine automations without building a new bespoke backend path each time.

Secondary user

A developer integrating an existing application with FlowForge through webhooks or a small set of supported actions.

Not for: teams that need branching, arbitrary code, enterprise connectors, or exactly-once business transactions.

Core jobs to be done

- When an event occurs, run a known sequence of actions.
- When a process must recur, run it on a schedule.
- When I need to investigate an incident, show me the exact execution and step that failed.
- When a transient failure occurs, retry safely without losing the execution record.
- When I need to verify a workflow, run it manually with test input.

Explicit non-goals for MVP

- Branching, loops, parallel fan-out/fan-in, sub-workflows, and human approval steps.
- A public marketplace, OAuth app installation system, or dozens of vendor integrations.
- Arbitrary user-provided JavaScript or shell commands.
- Distributed execution across many regions.
- Complex visual collaboration or real-time multi-user editing.

3. Main user journey

1. User signs in and lands on a dashboard with workflow counts, recent runs, and failure indicators.
2. User creates a workflow and gives it a name and optional description.
3. User selects a trigger: webhook, schedule, or manual.
4. User adds actions in order, configures each action, and maps trigger data into action fields.
5. User runs a test using sample input and inspects each step's output.
6. User reviews retry policy and trigger settings, then publishes the workflow.
7. An event, schedule, or manual run creates an execution.
8. The worker processes the steps and records statuses, timestamps, inputs, outputs, and errors.
9. User opens execution history, filters runs, and inspects a successful or failed run.
10. If permitted, user retries a failed step or starts a new execution with corrected input.

Terminology: a test run executes a snapshot of the draft with simulated side effects; a production run uses a published version; retry step continues the same execution; rerun creates a new linked execution.

Example workflow: contact form follow-up

Order	Node	Purpose	Example outcome
Trigger	Webhook received	Accept a contact form submission	Validated name, email, message
Action 1	Save contact	Persist the contact	Contact ID returned
Action 2	Send confirmation email	Acknowledge receipt	Provider accepted message
Action 3	Notify sales	Alert the team	Message posted to channel

The website sends the form payload to FlowForge. The customer sees the website's normal response; FlowForge executes the configured actions asynchronously. The UI exposes progress and final status to the operator.

4. Scope and feature priorities

Area	MVP	Later
Triggers	Webhook, schedule, manual	Polling and native app events
Flow shape	Linear sequential steps	Conditions, branches, parallelism

Area	MVP	Later
Actions	Database insert, HTTP request, email provider adapter, notification adapter (Slack-style webhook)	Delay action; broad integration catalog
Execution	Queued worker, per-step persistence, bounded retries	Worker pools, priorities, concurrency policies
Recovery	Retry eligible failed step in place; rerun creates a new execution linked to the original	Advanced replay, compensation, recovery policies
Editor	Add, remove, reorder, configure nodes; save and validate	Advanced graph editing and collaboration
Identity	Single-user/local development or simple workspace membership	Enterprise SSO and complex roles
Operations	Structured logs, health checks, queue visibility	Full metrics, tracing, alerting integrations

MVP acceptance criteria

- Invalid workflow configurations cannot be published.
- Published workflow versions are immutable snapshots for new executions.
- Each execution references the exact workflow version it used.
- Every step reaches a terminal, retrying or outcome_unknown state; the UI does not leave stale 'running' records indefinitely.
- A worker restart does not erase persisted execution state.
- Retries are bounded and use configurable delay/backoff.
- Webhook secrets are not shown in logs, and request bodies have size limits.
- The UI distinguishes a new run from retrying a failed execution.

5. UX and screen blueprint

5.1 Dashboard

- Summary cards: total workflows, successful runs, failed runs, average execution duration (last 7 days by default, test runs excluded).
- Recent executions list with workflow, status, start time, duration, and link to details.
- Primary action: Create workflow.
- Useful empty state: explain the three trigger types and offer a sample workflow.

5.2 Workflow list

- Columns: name, status (draft/published/paused), trigger type, last run, success/failure summary, updated time.
- Filters by status and trigger; search by workflow name.
- Actions: open, duplicate, pause/resume, archive. Destructive actions require confirmation.

5.3 Workflow creation wizard

1. Basics: name and description.
2. Trigger: choose webhook, schedule, or manual; show only relevant configuration fields.
3. Actions: add and configure supported action nodes.
4. Settings: retry policy, execution timeout, and failure behavior (MVP: stop on first permanent failure; no continue-on-error).
5. Review: validate the complete workflow and show warnings before publishing.

5.4 Visual builder

- Left panel: triggers/actions available for the current workflow.

- Canvas: ordered nodes with clear connector lines and step numbering.
- Right panel: configuration for the selected node.
- Top bar: workflow name, draft/published status, save state, Test, and Publish.
- Node states: not configured, valid, invalid, selected, running, succeeded, failed.
- Separate design-time states (not configured, valid, invalid, selected) from runtime states (pending, running, retrying, succeeded, failed, outcome_unknown), which appear only when a test or production execution is overlaid on the canvas.
- MVP interaction can be a constrained vertical sequence; avoid investing in a fully general graph editor.

5.5 Action configuration

- Use forms specific to each action instead of a generic JSON editor.
- Provide variable insertion from trigger data and previous-step outputs.
- Validate required fields inline and explain invalid variable paths.
- Show a sample input/output preview when possible.
- Mask secrets and credentials after saving.

5.6 Test run

- Allow the user to supply sample trigger input or use a saved example.
- Label the run as test and clearly indicate whether external side effects are real or simulated.
- Show step-by-step progress and inputs/outputs with secrets redacted.
- Provide a direct link to the execution details screen.

5.7 Execution list and details

- Filters: workflow, status, trigger type, time range; search by execution ID.
- Details: execution ID, workflow version, trigger source, start/end time, total duration, status.
- Timeline: trigger received, each action started/completed/failed, retries, and final state.
- Tabs or sections: Timeline, Input, Step details, Output, Logs.
- Retry actions are shown only when the execution state and step type make retry valid.

5.8 Failure recovery UX

- Display the failed step, concise error summary, attempt count, last attempt time, and retry eligibility.
- Before retry, explain whether successful prior steps will run again or be skipped.
- Require idempotency protections or explicit user confirmation before repeating non-idempotent side effects.
- Keep an audit event for who initiated a retry and when.
- Show outcome_unknown steps distinctly and offer three explicit choices: retry, mark as succeeded after the user verified the remote system, or mark as failed.

6. Trigger specifications

6.1 Webhook trigger

Purpose: start a workflow when an external application sends an HTTP request.

- Generate a high-entropy endpoint identifier and a separate secret or signature credential.
- Support a documented HTTP method and JSON payload in MVP.
- Validate request size, content type, authentication, and schema before accepting the event.
- Return a fast acknowledgement after the event is durably accepted, not after every workflow action finishes.
- Persist a trigger event and idempotency key where provided; deduplicate repeated deliveries within a defined window (default 24 hours).
- Keep raw payload retention configurable and redact sensitive values in logs.

- Deduplication scope is per webhook endpoint plus idempotency key within the window. Without a key, use the source system event ID if it provides one; otherwise accept the duplicate and document that callers should send idempotency keys.

6.2 Scheduled trigger

- Support interval-based schedules and a defined cron format; document the exact supported syntax.
- Store an explicit timezone, defaulting to UTC rather than relying on server-local time.
- Define daylight-saving behavior and what happens when a scheduled occurrence is missed during downtime.
- Use a scheduler process that creates executions; do not rely on an in-memory timer as the durable source of truth.
- Prevent duplicate creation for the same schedule and scheduled occurrence using a unique occurrence key.
- Examples: a daily 02:30 schedule in America/New_York fires once at 03:00 on the spring-forward day; on the fall-back day a 01:30 schedule fires once, at its first occurrence. Catch-up after downtime covers only the latest missed occurrence, and only if it is under 24 hours old.

6.3 Manual trigger

- Provide a Run now button for published workflows and a Test action for drafts.
- Validate the supplied input against the trigger's input schema.
- Record actor, request time, and whether the run is test or production.
- Manual execution must pass through the same durable execution pipeline as webhook and schedule triggers.

6.4 Future polling trigger

Polling should be added only after the core scheduler and execution pipeline are stable. A polling connector needs cursor/checkpoint storage, rate-limit handling, deduplication, and a policy for updates versus newly discovered records.

6.5 Future native app triggers

Native triggers usually require provider authentication, webhook subscription lifecycle, signature verification, event normalization, rate-limit management, and provider-specific operational handling. Treat each provider as a separate integration project.

7. Workflow definition and versioning

A workflow definition is a declarative document, not executable user code. It describes trigger configuration, ordered steps, mappings, and execution policy. Validate it when saved and again before publishing.

Concept	Rule
Draft	Editable working copy; not used by production triggers.
Published version	Immutable snapshot used by executions.
New publication	Creates a new version; does not silently mutate existing executions.
Execution	Pins one workflow version and records the trigger input.
Step mapping	References trigger data or outputs from earlier steps using validated paths.
Deletion	Archive by default; preserve execution history and referential integrity.

Validation rules

- Exactly one trigger per workflow.
- At least one action for a publishable workflow; trigger-only workflows are not supported in the MVP.
- Every action has a known type and valid configuration.

- Referenced variables exist and are available before the step that consumes them.
- Step order is valid; circular dependencies are impossible in the linear MVP.
- Timeouts and retry counts stay within server-defined limits.
- Secrets are stored separately from workflow JSON wherever possible.

8. System architecture

Recommended initial architecture: a modular monolith API plus a separate worker process, with PostgreSQL as the durable system of record and Redis/BullMQ as the delivery mechanism. Keep deployment simple while preserving clear module boundaries.

Component	Responsibilities	Must not do
Next.js web app	Workflow editor, dashboard, auth UI, execution views	Execute workflow steps in the browser
Node.js API	Auth, CRUD, validation, publish, webhook intake, run requests	Perform long-running workflow work inline
PostgreSQL	Durable definitions, versions, executions, step attempts, audit events	Rely on queue state as the only record
Redis + BullMQ	Deliver execution/step jobs, schedule retry delays, control worker concurrency	Become the source of truth for business state
Worker	Load pinned workflow version, execute steps, persist state transitions	Trust arbitrary workflow input or skip state checks
Scheduler	Find due schedule occurrences and create deduplicated executions	Use only process memory to remember future runs
External providers	Email, HTTP endpoints, notification systems	Assume every request is safe to repeat

Execution lifecycle

1. Trigger adapter authenticates and validates an event.
2. API or scheduler transactionally creates an execution record and an outbox event.
3. Outbox dispatcher publishes the event to BullMQ; repeat publishing is safe.
4. Worker claims the job and verifies the execution is still eligible to run.
5. Worker records step start, performs the action with a timeout, then records result.
6. If an action fails, classify the error as retryable or permanent.
7. Retryable errors are scheduled with bounded backoff; exhausted or permanent failures move to a terminal failure state.
8. After all steps succeed, mark the execution successful and emit an audit event.

Why use an outbox?

A database write and a Redis enqueue are not one atomic transaction. If the database commits but enqueueing fails, the execution could otherwise remain stranded. The outbox stores the intent to enqueue in the same PostgreSQL transaction as the execution; a dispatcher retries publishing until the message is delivered. Consumers must tolerate duplicate delivery.

Queue delivery semantics

Design for at-least-once job delivery. A job may be delivered more than once, so state transitions must be guarded and side effects must use idempotency support where available. Do not promise exactly-once external side effects.

Architecture diagram

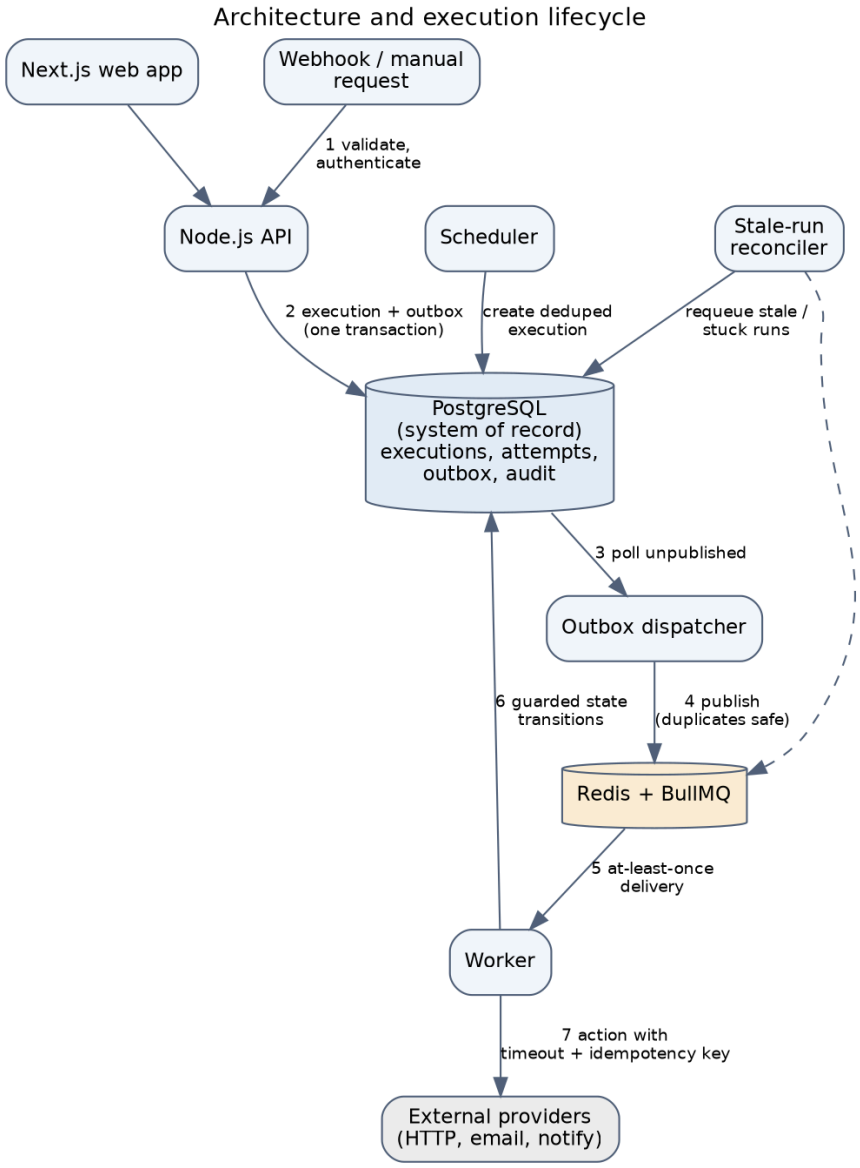


Figure 1. Architecture and execution lifecycle (a simplified view of the eight steps in section 8).

9. Suggested technology stack

Layer	Choice	Reason / boundary
Frontend	Next.js + TypeScript	Application shell, routing, server-rendered pages where useful
UI	shadcn/ui component library; React Flow deferred	The MVP builder is a plain ordered list; add React Flow only if a graph editor is later needed
API	Node.js + TypeScript	Keep API and worker contracts typed and shared
Validation	Zod or equivalent schema validation	Validate API inputs and workflow definitions at trust boundaries
Database	PostgreSQL	Transactions, relational integrity, JSONB for bounded flexible configuration
Queue	Redis + BullMQ	Retries, delayed jobs, concurrency controls

Layer	Choice	Reason / boundary
Testing	Unit tests + PostgreSQL/Redis integration tests + browser E2E	Verify failure behavior, not just happy paths
Local development	Docker Compose	Reproducible API, worker, PostgreSQL, and Redis environment
Observability	Structured logs and health endpoints	Correlate each log with execution ID and step attempt

Avoid introducing Kafka, Kubernetes, microservices, or a generic plugin marketplace for the MVP. They add operational surface area before the product's central execution semantics are proven.

10. Data model blueprint

Entity	Key fields	Purpose
Workspace	id, name, created_at	Ownership boundary for workflows and credentials
User / Membership	id, email; workspace_id, role	Identity and workspace authorization
Workflow	id, workspace_id, name, status, current_version_id	Stable identity and lifecycle of a workflow
WorkflowVersion	id, workflow_id, version_number, definition, published_at	Immutable trigger, step, mapping, and policy snapshot
Credential	id, workspace_id, provider, encrypted_secret, metadata	Secrets and provider credentials stored separately
Execution	id, workflow_version_id, trigger_type, status, input, started_at, finished_at, idempotency_key	One attempted workflow run
StepExecution	id, execution_id, step_key, status, input, output, error, started_at, finished_at	Current result/state for a step within an execution
StepAttempt	id, step_execution_id, attempt_number, status, error, timing	History of individual attempts
Schedule	id, workflow_id, timezone, schedule_spec, enabled	Derived index of the published definition; unique on (schedule_id, scheduled_for)
TriggerEvent	id, workflow_id, source_key, received_at, payload_digest	Webhook receipt and deduplication record
OutboxEvent	id, aggregate_type, aggregate_id, event_type, payload, published_at	Reliable publication of queued work
AuditEvent	id, workspace_id, actor_id, action, target_id, created_at, metadata	Track publish, pause, retry, credential changes

Data integrity requirements

- Use foreign keys and indexes for workspace, workflow, status, and time-based execution queries.
- Use unique constraints for workflow version numbers and deduplication keys within their intended scope.
- Keep step attempt history separate from the current step summary.
- Avoid storing secrets in workflow definitions, inputs, outputs, or error messages.
- Set a retention policy for large payloads and execution logs.

Execution statuses

Object	Suggested states	Meaning
Workflow	draft, published, paused, archived	Whether new production executions may be created
Execution	queued, running, retrying, succeeded, failed, cancelled	Overall run lifecycle
Step	pending, running, retrying, succeeded, failed, skipped, outcome_unknown	Individual step lifecycle
Attempt	running, succeeded, failed, timed_out	One attempt to perform an action

Define allowed state transitions centrally. Terminal states should not be overwritten by a late duplicate queue delivery without an explicit recovery operation. The only permitted exit from failed is a recovery operation (retry step or rerun execution), which must be audited and guarded by a retry-request idempotency key.

State machine diagram

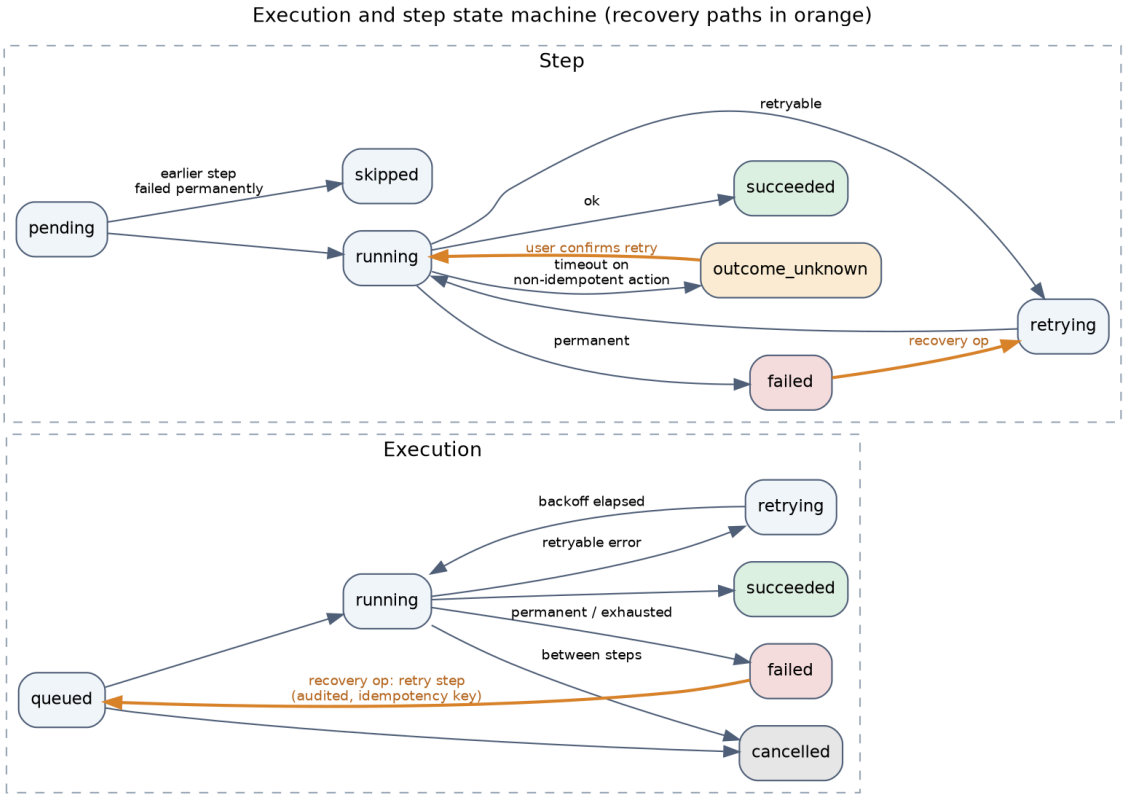


Figure 2. Execution and step states; orange edges are recovery operations only.

State transition table

Object	Transition	Actor and guard	Audit
Execution	queued to running	Worker; lease acquired	No
Execution	running to retrying, then back to running	Worker; retryable error, attempts left, then backoff elapsed	Attempt record
Execution	running to succeeded	Worker; every step succeeded	Yes
Execution	running to failed	Worker; permanent error, attempts exhausted, or a step in outcome_unknown	Yes
Execution	failed to queued	Editor; recovery operation with a RetryRequest idempotency key	Required
Execution	queued or running to cancelled	Editor; queued, or between steps only	Required
Step	pending to running to succeeded, failed or retrying	Worker; lease token matches	Attempt record
Step	running to outcome_unknown	Worker; timeout on an action that is not safe to repeat	Yes
Step	failed or outcome_unknown to retrying	Editor; recovery operation	Required

Object	Transition	Actor and guard	Audit
Step	outcome_unknown to succeeded or failed	Editor; manual resolution stored as a resolution record (actor, reason, time); attempt history is never edited	Required
Step	pending to skipped	System; an earlier step failed permanently, or the execution was cancelled. Skipped steps are retried only through the upstream failed step	No
Forbidden	Any exit from succeeded or cancelled; failed straight to succeeded; any change without a matching lease token	Applies to system actors too, including the reconciler	Alert

- An execution with a step in outcome_unknown is moved to failed with a failure reason of outcome_unknown, so it appears in the recovery list. Cancelled is an execution state only; pending steps become skipped.
- Marking an outcome_unknown step as succeeded writes a resolution record and sets the step to succeeded with resolved_manually set. The original attempt stays recorded as timed out.

Retry step vs. rerun

Aspect	Retry step	Rerun
Execution	Same execution	New execution with rerun_of_execution_id
Workflow version	Same pinned version	Original version by default; current published version only on explicit choice
Idempotency	Same stable per-step keys	New execution key, so it is not deduplicated against the original
Successful steps	Not repeated	All steps repeat, so the UI must warn about side effects
Typical use	Transient provider failure, or a resolved outcome_unknown	Corrected input or a changed workflow

Data model additions

- WorkflowDraft (workflow_id, definition, revision): drafts need a home separate from immutable versions; add a "published with unpublished changes" indicator.
- WebhookEndpoint (id, secret_hash, previous_secret_hash, rotated_at): supports rotation with a short overlap window.
- RetryRequest (id, execution_id, step_key, idempotency_key, actor_id): prevents duplicate recovery jobs.
- StepExecution: add worker_id and lease_expires_at so the stale-run reconciler has something to query. Execution: add trigger_event_id and is_test.
- Schedule (canonical rule): references workflow_id, never a fixed version. The published definition holds the schedule configuration and the Schedule table is a derived index for the scheduler. The unique occurrence key is (schedule_id, scheduled_for), and each fired occurrence creates an execution that pins the current published workflow_version_id.
- Rerun semantics: retry step continues the same execution and moves it from failed back to queued. Rerun creates a new execution with rerun_of_execution_id set and a new idempotency key (reusing the old key would deduplicate against itself).
- Webhook endpoints: bind to the workflow, not to a version, so the URL survives republishing. Keep secret material in the endpoint record with rotation metadata. Changing trigger settings stored in the definition requires publishing a new version; rotating a secret does not.
- Credential: workflow definitions reference credential_id only. Check workspace ownership of the referenced credential at publish and at run time, and block deleting a credential that a published workflow uses.

- Roles: define owner, editor and viewer for the MVP. Editors can publish and retry; owners can rotate credentials; viewers are read-only. The MVP UI exposes one workspace per user, but every query still carries workspace_id.
- OutboxEvent: delete published rows after a fixed period (for example 7 days) so the table does not grow without bound.
- Unique constraints: (execution_id, step_key); (step_execution_id, attempt_number); (schedule_id, scheduled_for); (endpoint_id, idempotency_key) within the dedupe window; the RetryRequest idempotency key; (workflow_id, version_number).
- Outbox status model: pending, published, failed (will retry), dead_letter after a maximum number of attempts. Store attempt_count and next_attempt_at, and alert on dead_letter rows.
- Test runs pin a draft revision, set is_test, and cannot be promoted into production executions.

11. Action contract and execution behavior

Every action adapter should follow one shared conceptual contract: validate configuration, resolve inputs, execute with a deadline, return a structured result, classify failures, and expose whether repeating the action is safe.

Action	MVP behavior	Important safeguards
HTTP request	Call a configured HTTP endpoint with selected method, headers, and body	Block private/internal network destinations unless explicitly allowed; limit response size; timeout; redact headers
Send email	Send through one provider adapter or local test adapter	Provider credentials; retry classification; provider idempotency if available
Database insert	Insert into a FlowForge-managed contacts table for the demo	Parameterized queries; validate columns; avoid arbitrary SQL
Notification	Send a message to one supported destination	Credential isolation; rate limits; retry safety
Delay	Optional later step that waits until a specified duration	Persist scheduled continuation rather than holding a worker open

Variable mapping

Users should be able to map values from the trigger input and prior step outputs. The UI should present a picker rather than requiring users to memorize paths. Validate mappings at publish time and show missing values before execution.

Side effects and idempotency

- Generate a stable idempotency key per execution and step where the provider supports it.
- For database writes, enforce a suitable unique business key or use an idempotent upsert.
- Do not automatically rerun all successful steps when retrying one failed step unless the workflow's policy explicitly says so.
- Explain that a timeout does not prove the remote operation failed; the provider may have completed it without returning a response.
- Add a step status outcome_unknown for timeouts and lost responses on non-idempotent actions; the UI must require explicit confirmation before retrying such a step.
- Each action adapter declares its output schema, redaction rules, error classification and safe_to_repeat flag, and the registry rejects adapters that omit them.

Error taxonomy

Error class	Examples	Retry behavior	UI treatment
Validation error	Invalid mapping or configuration	None	Fail; fix the workflow
Permanent provider error	4xx rejection	None	Fail with sanitized detail
Transient provider error	5xx, connection reset	Automatic, backoff with jitter	Retrying with attempt count

Error class	Examples	Retry behavior	UI treatment
Rate limited	429	Automatic, honor Retry-After	Retrying with next attempt time
Timeout (outcome unknown)	No response within the deadline	Automatic only if safe_to_repeat	outcome_unknown with resolve options
Internal infrastructure error	Database or Redis unavailable	System retries the job; no attempt consumed	Operator alert

Proposed default limits

These are starting values to tune, not requirements; the point is that every "server-defined limit" in this document has a number.

- Steps per workflow: 10. Webhook body: 256 KB. HTTP action response: 1 MB.
- Step timeout: 30 s default, 120 s maximum. Execution timeout: 10 minutes.
- Retries: 3 attempts by default, 5 maximum. Backoff: 5 s base, 5 min cap, with jitter.
- Webhook deduplication window: 24 hours. Execution payload retention: 30 days. Concurrency: 1 active run per workflow by default.

Variable mapping rules

- Use a single syntax such as {{trigger.body.email}} and {{steps.save_contact.output.id}}. No expressions or functions in the MVP. Reference steps by stable step keys (lowercase slugs fixed at creation), never by display name.
- A path that cannot resolve at publish time is a validation error. A value that is null at runtime is a permanent step failure unless a default is configured.
- Strip CR/LF from mapped values used in email headers. HTTP actions must not follow redirects by default, and must re-validate the destination on every hop if they do.

12. API surface blueprint

These are conceptual endpoint groups, not a final wire contract. The MVP uses REST, and the execution lifecycle must stay independent from the frontend transport.

Capability	Operations	Notes
Workflows	List, create, update draft, validate, publish, pause, resume, archive	Enforce workspace access on every operation
Workflow versions	Read version details and published snapshots	Published snapshots are immutable
Triggers	Receive webhook; inspect or rotate webhook credential	Rate-limit and authenticate intake
Executions	List, read details, request manual run, retry eligible failure, cancel where safe	Separate retry from new execution
Schedules	Create, update, enable, disable	Persist timezone and schedule occurrence identity
Actions	List available action types and validate configuration	Use a server-maintained action registry
Credentials	Create, test, rotate, delete	Never return plaintext secret after initial save
Operations	Health/readiness, queue diagnostics, worker heartbeat	Restrict operational endpoints

API rules

- Use consistent pagination, filtering, sorting, and error envelopes.
- Return a correlation/request ID for support and log correlation.

- Use optimistic concurrency or version checks when updating drafts.
- Apply authorization at the server boundary; hiding UI controls is not authorization.
- Webhook intake should return promptly after durable acceptance and clearly distinguish rejected requests from accepted events.
- Include endpoints to run a test, rerun an execution, resolve an outcome_unknown step, and read the audit log. Retry and rerun requests take an idempotency key. Document webhook signature verification with headers, algorithm, timestamp tolerance and a worked example.

13. Security and abuse prevention

- Authentication: use a maintained authentication solution; do not invent a password/session protocol.
- Authorization: scope every workflow, execution, schedule, and credential query to the user's workspace.
- Webhook security: high-entropy secrets or signed requests, rotation, replay/deduplication controls, request-size limits, and rate limiting.
- Secrets: encrypt at rest, restrict access to worker code, redact logs, and never expose stored values in API responses.
- SSRF protection: HTTP action URLs can be abused to reach localhost, cloud metadata endpoints, or private networks; validate destinations and re-check resolved addresses to resist DNS rebinding.
- Input validation: treat webhook payloads and action outputs as untrusted data.
- Resource limits: maximum steps, workflow size, execution duration, response body size, retries, and concurrent runs.
- Audit: record publish, pause, credential rotation, retry, and other sensitive operations.
- Encrypt secrets with envelope encryption: a rotatable master key, with a key ID stored beside each ciphertext. Make audit events append-only by denying update and delete to the application role. Add SSRF regression tests for redirects, IPv6 and DNS rebinding.
- Data minimization: allow payload retention to be limited; redact emails, tokens, and personal data from logs.
- Isolation: do not execute arbitrary user code in the API or worker process.

14. Reliability and recovery design

Failure case	Expected behavior
API crashes before database commit	No accepted execution; sender may retry the webhook.
Database commits but queue publish fails	Outbox dispatcher publishes later.
Queue delivers duplicate job	Worker checks persisted state and avoids invalid duplicate transitions.
Worker crashes during an action	Lease/timeout recovery marks stale work for controlled retry; side effects may need idempotency.
External provider returns 429 or transient 5xx	Retry with bounded exponential backoff and jitter, respecting provider guidance.
Provider returns permanent validation error	Fail promptly with sanitized actionable detail.
Schedule process restarts	Reconcile due schedule occurrences from durable schedule records.
User pauses workflow	Stop creation of new production runs; in-flight runs finish and queued runs are held until resume (see Reliability, webhook and scheduling policies).
Workflow is edited after publication	Existing execution remains pinned to its original version.
Retry requested twice	Use a retry-request idempotency key or state guard to prevent duplicate recovery jobs.

Retry policy

- Make maximum attempts, base delay, maximum delay, and timeout configurable within safe limits.
- Use exponential backoff with jitter for transient failures.

- Distinguish action attempt retry from whole-workflow rerun.
- Avoid retrying permanent errors such as invalid configuration without user intervention.
- Provide an explicit cancellation policy; cancellation cannot always undo an external side effect already in progress.

Additional infrastructure failure cases

- Redis data loss: the reconciler rebuilds the queue by re-enqueueing queued and lease-expired executions from PostgreSQL.
- Outbox poison message: after the maximum attempts the row becomes dead_letter and raises an alert instead of retrying forever.
- Database failover: workers stop claiming work until they reconnect; in-flight leases expire and the reconciler recovers them.
- Clock skew: leases, schedules and deadlines use database time, not worker clocks.
- Duplicate scheduler polling: the unique occurrence key plus an advisory lock keep a single creator.

Reliability, webhook and scheduling policies

- Queue model: one BullMQ job per execution continuation, with jobId set to execution_id:run_sequence so a later re-enqueue is not deduplicated against a retained completed job. A continuation runs steps until the execution finishes, fails, or needs a delayed retry, which is enqueued as the next continuation. The outbox dispatcher claims rows with FOR UPDATE SKIP LOCKED so several dispatchers can run safely, retries failed publishes with backoff, and marks a row dead_letter after a maximum number of attempts. The reconciler also re-enqueues executions stuck in queued, in case Redis loses data.
- Webhooks: HMAC-SHA256 over timestamp plus body with a 5-minute tolerance. A repeat delivery with the same idempotency key inside the window returns 200 with the original execution ID and creates nothing. A paused workflow rejects intake with 409 so the sender retries.
- Schedules: evaluate in the stored IANA timezone. A skipped local time fires at the next valid time; a repeated hour fires once. After downtime, create at most one catch-up run for the latest missed occurrence.
- Pause and cancel: pausing stops new runs, lets in-flight runs finish, and holds queued runs until resume. Cancel applies to queued runs or the gap between steps; a running attempt completes and is recorded.
- Test runs: execute against a snapshot of the draft, flag is_test, simulate side effects by default with a per-step opt-in for real ones, and exclude them from dashboard metrics. Add a test/production filter to the execution list.
- Audit: when a retry or run is initiated by a webhook or the scheduler, record a system actor rather than leaving actor_id empty.
- Backpressure: with one active run per workflow, bursts queue up. Cap queued executions per workflow (for example 1,000) and reject further intake with 429, and apply a per-workspace intake rate limit.
- Webhook status codes: 202 for a newly accepted event, 200 for a duplicate, 409 for a paused workflow, 429 when over a limit.
- Execution timeout vs. the future Delay action: model a delay as a persisted continuation that does not count against active run time, and cap total elapsed time separately.
- Worker claim protocol: claim the execution, acquire a database lease with a unique lease token, heartbeat while working, apply every state change with a guard on the lease token, and release the lease at the end. Lease duration must exceed the maximum step timeout plus a heartbeat margin. The reconciler re-enqueues only when the lease has expired and the execution is still in a recoverable state. BullMQ stalled-job detection is secondary: the database lease and guarded transitions are authoritative, and the BullMQ lock duration is set above the step timeout.
- Duplicate delivery vs. duplicate side effect: the lease and guards make duplicate queue delivery harmless to FlowForge state, but they cannot stop an external system from seeing a repeated request after a crash. That is why every side-effecting step carries a stable idempotency key, and why uncertain outcomes become outcome_unknown instead of being retried blindly.

15. Testing strategy

Test level	What to verify
Unit	Definition validation, variable mapping, state transitions, retry classification, schedule parsing
Integration	PostgreSQL transactions, outbox dispatcher, Redis/BullMQ behavior, worker persistence
Contract	Each action adapter validates inputs and returns normalized outputs/errors
End-to-end	Create workflow, test, publish, trigger run, inspect success and failure, retry eligible step
Resilience	Kill worker during execution, make provider time out, duplicate webhook delivery, interrupt queue publishing
Security	Workspace authorization, secret redaction, request limits, SSRF protections, replay/deduplication

Minimum critical test scenarios

1. Valid webhook creates exactly one accepted execution for one idempotency key.
2. Invalid or unauthenticated webhook is rejected and does not enqueue work.
3. Worker executes steps in order and records step outputs.
4. Transient failure retries and eventually succeeds.
5. Permanent failure stops the workflow with a useful sanitized error.
6. Worker restart does not erase state or silently strand the run.
7. A published version remains unchanged after a draft edit.
8. A user cannot access another workspace's workflow, execution, or credential.
9. Manual retry does not accidentally repeat previously successful non-idempotent steps.
10. A due schedule occurrence is not created twice during scheduler restart or concurrent polling.

- A step that times out on a non-idempotent action becomes outcome_unknown and is not retried automatically.
- Pause, resume and cancel behave as specified, including webhook intake returning 409 while paused.
- Schedules behave correctly across a DST change, and a missed occurrence after downtime produces at most one catch-up run.
- A rotated webhook secret works during the overlap window and is rejected afterwards.
- Two simultaneous recovery requests for the same failed step create one recovery, not two.
- Run unit and integration suites in CI on every pull request, including migrations applied to an empty database.
- A crash between the database commit and the Redis publish leaves the execution to be published by the dispatcher.
- After a Redis data wipe, queued and lease-expired executions are re-enqueued by the reconciler.
- A lease expiring while a slow step is still running does not produce two concurrent writers.
- DNS rebinding and redirect-to-private-address attempts are blocked by the HTTP action.
- Manually resolving an outcome_unknown step preserves the original attempt history.

16. Observability and operations

- Use structured logs with request ID, workspace ID, workflow ID, execution ID, step ID, and attempt number.
- Track counters: executions accepted, succeeded, failed, retried, and rejected; queue depth; worker heartbeat; step duration.
- Track latency from trigger receipt to queue start and from queue start to execution completion.
- Provide health checks for API, database connectivity, queue connectivity, and worker liveness.
- Add a stale-run reconciler for executions that remain running beyond a defined lease/timeout.
- Keep logs useful but avoid logging secrets or entire personal-data payloads by default.
- Suggested metrics: flowforge_executions_total by status, flowforge_step_duration_seconds, flowforge_queue_depth, flowforge_outbox_lag_seconds, flowforge_worker_heartbeat_age_seconds. Suggested

alerts, as starting values: outbox lag over 60 s, any execution running past its lease for 5 minutes, queue depth rising for 10 minutes, failure rate above 20% over 15 minutes.

17. Deployment and local development

Use a simple Docker Compose setup for local development. Separate the API and worker processes even if they share one codebase.

Service	Purpose	Local requirement
web	Next.js UI	Browser-accessible frontend
api	HTTP API and webhook intake	Environment config and database connection
worker	BullMQ consumer and action executor	Queue connection and worker heartbeat
scheduler	Creates due schedule executions	Can initially be a separate process or isolated module
postgres	Durable data store	Migration and seed support
redis	Queue transport	Persistence/config appropriate to local development
mail test adapter	Safe email testing	Use a sandbox provider or local mail catcher

Environment configuration

Document required environment variables, credential generation, migration commands, seed data, health checks, and how to run tests. Keep real secrets out of version control. Include a one-command local startup path where practical.

Production readiness checklist (aspirational for the portfolio MVP)

- TLS at ingress; secure cookie/session configuration.
- Database backups and tested restore process.
- Redis availability and a clear queue recovery procedure.
- Worker concurrency and per-workspace rate limits.
- Secret rotation procedure and audit trail.
- Retention/cleanup jobs for old payloads and execution details.
- Error reporting and alerting for queue lag, repeated failures, and stale executions.

18. Implementation roadmap

Phase	Deliverable	Exit criteria
0. Foundation	Repo, lint/test setup, Docker Compose, migrations, auth boundary	A developer can start the app locally and run tests
1. Thin vertical slice	Create one webhook workflow (also runnable with the manual trigger) with one action and one execution details page	An event produces a persisted, visible execution
2. Execution engine	Step state machine, queue worker, attempts, outputs, timeouts	Ordered execution and failure states work reliably
3. Reliability	Outbox, deduplication, retry/backoff, stale-run recovery	Crash and duplicate-delivery tests pass
4. Product UX	Dashboard, workflow list, linear builder, variable mapping, publish/version flow	A user can configure and publish without editing database records
5. More triggers/actions	Schedule trigger; HTTP (with SSRF protections), email and notification actions	All trigger types use the same execution pipeline
6. Finish and demonstrate	E2E suite, seed demo, docs, architecture diagram, recorded demo	A reviewer can run it locally and understand trade-offs

Recommended order

Build the backend vertical slice before polishing the canvas. The product's strongest engineering signal is reliable execution and recovery, not visual complexity. Use a simple linear editor until the underlying model is stable.

Roadmap changes (applied above): the manual trigger is in Phase 1 so the engine can be exercised early, SSRF protections ship with the HTTP action, and React Flow is deferred because the MVP builder is a plain ordered list. Effort estimates per phase remain an open decision.

19. Definition of done

- Core user journeys work end to end without manual database edits.
- Migrations, seed data, local setup, and test commands are documented.
- Unit, integration, and key E2E tests pass.
- Execution state transitions and retry semantics are documented.
- No secrets appear in logs or API responses.
- Failure cases are demoable and produce useful diagnostic information.
- Architecture diagram, data model, API overview, and known limitations are included in the repository.
- The demo includes a happy path and at least one controlled failure/recovery path.

20. Portfolio presentation

What to demonstrate

1. Create a contact-form workflow with a webhook, database action, email action, and notification action.
2. Run a successful test and show persisted step-by-step execution history.
3. Simulate a temporary provider failure and show bounded retry behavior.
4. Restart a worker during a controlled run and demonstrate recovery behavior.
5. Show how duplicate webhook delivery is handled.
6. Explain the outbox pattern, workflow version pinning, idempotency limitations, and why the system does not claim exactly-once side effects.

Suggested repository documentation

- README: problem, demo GIF/video, local startup, architecture summary.
- Architecture decision records: outbox, queue choice, workflow versioning, retry semantics, secret storage.
- System design document: component diagram, state machines, data model, failure matrix.
- API documentation: auth, webhook signature, error handling, pagination, retry semantics.
- Operations guide: local troubleshooting, queue inspection, database reset, backup/restore assumptions.

21. Risks, trade-offs, and open decisions

Decision	Recommendation	Why / revisit when
REST or GraphQL API	REST with JSON; record the decision in an ADR	Do not let transport choice delay execution semantics
Visual editor	Constrained linear flow in MVP	General graphs create complexity in validation and recovery
Email provider	Resend as the real provider, plus a local mail catcher for tests	Multiple providers are not core proof of value
Workflow concurrency	Start with bounded per-workflow concurrency	Prevent overlapping scheduled runs from causing accidental duplication
Payload retention	Store limited/redacted payloads with configurable retention	Protect privacy and control database growth

Decision	Recommendation	Why / revisit when
Retry scope	Retry failed step when safe; separate whole-run replay	External side effects make naive replay dangerous
Multi-tenancy	Model workspace ownership from the start, keep roles simple	Do not build enterprise tenancy features before needed
Scheduling	Persist schedule definitions and deduplicated occurrences	In-memory timers are not durable

Positioning

FlowForge is a small, self-hostable workflow engine for technical operators who need execution transparency and safe recovery. It is not a Zapier or n8n replacement: it does less, but every run is inspectable and every failure is recoverable without silent repeats. Sections describing launch operations, such as the production readiness checklist, are aspirational for the portfolio MVP.

Final recommendation

Treat FlowForge as an execution and observability product with a visual configuration layer. Keep the MVP intentionally narrow: three triggers, a few actions, sequential execution, immutable published versions, persisted attempts, retries, and a clear history screen. Deliver that well before adding advanced graph features or many integrations.

Review changelog

- Added architecture and state-machine diagrams, a state transition table, a retry-vs-rerun table and an error taxonomy.
- Defined the queue, lease and claim model, schedule and webhook source-of-truth rules, default limits, variable mapping rules and policy defaults.
- Reworded the measurable targets to match at-least-once delivery.
- Fixed numbered lists that did not restart.
- Closed the open review notes: Phase 1 includes the manual trigger, REST and Resend chosen, shadcn/ui named, Delay listed as post-MVP, positioning written as a statement.
- Layout: repeating table header rows, rows no longer split across pages, page numbers in the footer, document properties set.

Open decisions

- Effort estimate for each roadmap phase.
- Benchmark environment and method behind the proposed measurable targets.
- Whether to add a table of contents once the document is final.